

42 SCHOOL • FOUNDATIONAL C CURRICULUM

Libft

*The Complete Textbook — Header, Build System & 43
Functions Explained*

Author `iismail-`

Campus `42 Warsaw`

Functions Covered `43 / 43`

Norm Status `100% Compliant`

Build Status `libft.a — Verified`

Memory Audit `Valgrind Clean`

Table of Contents

PART 1 — THE BLUEPRINT

Header Guards	— why	<code>#ifndef</code>	/	<code>#define</code>	exist	1.1
The <code>t_list</code> Structure	— anatomy of a node					1.2
Prototypes → Implementations	— how the compiler links them					1.3

PART 2 — THE AUTOMATOR

Makefile Internals	— targets, dependencies, recipes					2.1
Core Variables	— CC, CFLAGS, AR, SRCS, OBJS					2.2
The Pipeline	— <code>.C</code> → <code>.o</code> → <code>.a</code>					2.3
Relinking	— the rule that catches careless students					2.4

PART 3 — THE LIBRARY DEEP-DIVE

3.1 Character Classification & Case	— 7 functions	<code>ft_isalpha</code> ... <code>ft_tolower</code>
3.2 Memory Primitives	— 6 functions	<code>ft_memset</code> ... <code>ft_memmove</code>
3.3 String Inspection & Comparison	— 8 functions	<code>ft_strchr</code> ... <code>ft_atoi</code>
3.4 Heap Allocation	— 2 functions	<code>ft_calloc</code> , <code>ft_strdup</code>
3.5 String Construction	— 5 functions	<code>ft_substr</code> ... <code>ft_itoa</code>
3.6 Function Pointers & I/O	— 6 functions	<code>ft_strmapi</code> ... <code>ft_putnbr_fd</code>
3.7 The Linked List Engine	— 9 functions	<code>ft_lstnew</code> ... <code>ft_lstmap</code>

PART 1 • THE BLUEPRINT

libft.h

Every library begins as a contract. Before a single algorithm runs, the header file declares *what exists* — without saying how. This chapter explains why that separation matters, and how three small lines of preprocessor syntax quietly protect an entire multi-file project from itself.

1.1 — Header Guards: Why #ifndef Exists

Look at the top and bottom of your header:

```
#ifndef LIBFT_H
# define LIBFT_H

... entire header body ...

#endif
```

This is called an **include guard**, and it solves a very specific problem that only appears once your project has more than one `.c` file: **double inclusion**.

Imagine `ft_strtrim.c` includes `libft.h`, and that header also gets pulled in indirectly by another file it depends on. Without a guard, the preprocessor would paste the entire contents of `libft.h` into the translation unit *twice*. The compiler would then see the `t_list` struct declared twice and throw a **redefinition** error — a project that compiled fine yesterday suddenly refuses to build the moment a second file starts including the header indirectly.

The guard works through simple conditional compilation logic:

1. **First inclusion:** the macro `LIBFT_H` is not yet defined, so `#ifndef LIBFT_H` ("if not defined") evaluates true. The preprocessor immediately runs `#define LIBFT_H`, marking the guard as "used," then includes everything up to `#endif`.
2. **Every subsequent inclusion in the same translation unit:** `LIBFT_H` is now defined, so `#ifndef` is false. The preprocessor skips straight to `#endif`, contributing nothing. The struct, the typedef, and all 43 prototypes are declared exactly once — no matter how many files transitively include the header.

Why the name `LIBFT_H` specifically? By convention, the guard macro matches the filename in uppercase with the dot replaced by an underscore. This isn't enforced by the compiler — it's a human convention that prevents *two different headers in the same project* from accidentally sharing a guard name and silently shadowing each other.

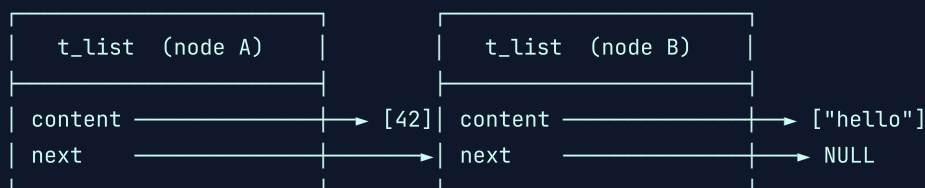
INCLUDES INSIDE THE GUARD

```
#include <stdlib.h>
#include <unistd.h>
```

`<stdlib.h>` is required because `malloc`, `free`, and `NULL` are used throughout the allocation-heavy functions (`ft_calloc`, `ft_split`, `ft_substr`, and the entire linked-list module). `<unistd.h>` is required because it's the only standard header that declares `write()` — the single system call underneath every one of the `ft_put*_fd` functions. Both headers are themselves guarded internally by the C library, so nesting them inside `libft.h`'s own guard is completely safe even if a consuming program also includes them directly.

1.2 — Anatomy of a Node: the `t_list` Structure

```
typedef struct s_list
{
    void          *content;
    struct s_list *next;
} t_list;
```



TWO NODES CHAINED BY 'NEXT'. 'CONTENT' IS OPAQUE — IT POINTS ANYWHERE.

Two fields, two very different jobs:

FIELD	TYPE	ROLE
<code>content</code>	<code>void *</code>	A generic pointer. Because <code>void *</code> carries no type information, a single <code>t_list</code> node can hold an <code>int *</code> , a <code>char *</code> , a custom <code>struct</code> — anything. The struct itself

never needs to know what it's storing; that responsibility is deferred entirely to the caller, and to the function pointers (`del` , `f`) passed into functions like `ft_lstclear` and `ft_lstmap` .

`next` `struct` A self-referential pointer — the struct contains a pointer to *its own type*. This is the mechanism that turns isolated nodes into a chain. `NULL` in this field is the list's own sentinel: it means "there is nothing after me," and every traversal function in this library (`ft_lstsize` , `ft_lstlast` , `ft_lstiter` ...) relies on that convention to know when to stop.

`s_list`

`*`

Why `struct s_list *next` and not `t_list *next` ? At the point where `next` is declared, the `typedef` hasn't finished yet — the compiler is still *inside* the definition of `struct s_list` , so the name `t_list` doesn't exist as a usable type until the closing `}` and the trailing `t_list;` are reached. The struct's own tag name, `s_list` , is always visible inside its own body (this is one of the few places C allows a genuinely recursive, self-referential type), so that's what must be used.

1.3 — How Prototypes Become Implementations

A prototype like

```
char    *ft_substr(char const *s, unsigned int start, size_t len);
```

is a **promise**, not a function. It tells the compiler three things: the function's name, the number and type of its parameters, and its return type — enough for the compiler to check that every *call site* is using it correctly, without yet knowing what the function actually does.

The real machinery happens at **link time**, after every `.c` file has been compiled independently into a `.o` object file:

1. When `gcc` compiles `ft_strjoin.c` and encounters a call to `ft_substr(...)` , it checks the prototype from `libft.h` , confirms the call is type-correct, and emits machine code containing an **unresolved symbol reference** to the name `ft_substr` .
2. Separately, `ft_substr.c` compiles into `ft_substr.o` , which contains the actual machine code for the function body, exported under the symbol `ft_substr` .
3. When these object files are archived together into `libft.a` via `ar` , and a program later links against that archive, the linker matches every unresolved reference to its defining symbol — stitching `ft_strjoin.o` 's *call* to `ft_substr.o` 's *body*.

This is precisely why a **signature mismatch** between the header and the implementation is a compile-time error rather than a silent bug: if `libft.h` promises `const void *src` but the `.c` file defines the parameter as `const char *src`, the compiler catches the contradiction the moment it compiles that specific file — `error: conflicting types` — long before the linker ever runs. The header is the single source of truth every `.c` file is measured against.

PART 2 • THE AUTOMATOR

Makefile

A Makefile is a dependency graph disguised as a script. This chapter explains what actually happens when you type `make`, why each variable exists, and — critically for 42 evaluation — how this specific Makefile architecture guarantees that running `make bonus` never wastes time recompiling work that's already done.

2.1 — How Makefiles Work Under the Hood

`make` is not a compiler. It is a **dependency resolver**. Every rule in a Makefile has the same three-part shape:

```
target: prerequisite1 prerequisite2
    recipe-line-1
    recipe-line-2
```

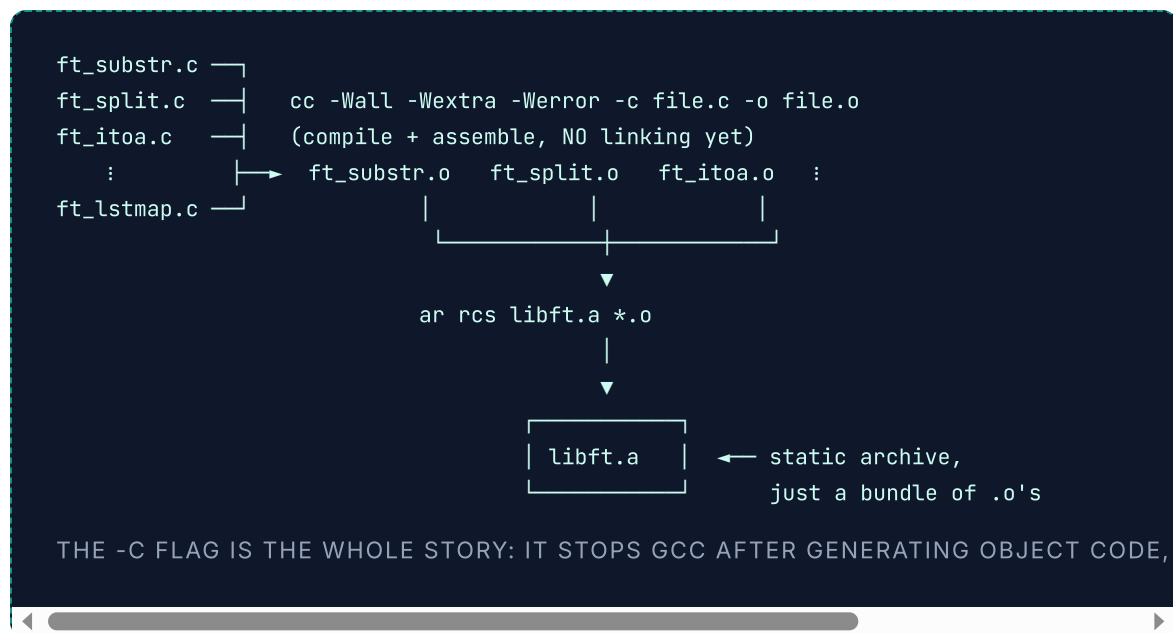
Read as: "to produce **target**, every **prerequisite** must exist and be newer than the target. If any prerequisite is missing or was modified more recently than the target, run the recipe." This is the entire engine. `make` compares file **modification timestamps** on disk — nothing more exotic than that — which is exactly why the tab character before each recipe line is mandatory (not stylistic): it's how `make`'s parser distinguishes a recipe line from a new rule.

2.2 — Core Variables, Explained

VARIABLE	VALUE	PURPOSE
----------	-------	---------

NAME	libft.a	The final artifact this Makefile exists to produce. Used as the default target so <code>make</code> with no arguments builds exactly this.
CC	cc	The compiler invoked for every translation unit. Using the variable instead of hardcoding <code>gcc</code> everywhere means switching compilers is a one-line change.
CFLAGS	-Wall -Wextra -Werror	<code>-Wall</code> enables the broad "commonly useful" warning set; <code>-Wextra</code> adds stricter checks <code>-Wall</code> omits (unused parameters, sign-compare issues); <code>-Werror</code> escalates every warning into a hard compile failure. Together they turn latent bugs — like the <code>const char *</code> vs <code>const void *</code> mismatch this project once had — into errors you cannot ignore.
AR	ar rcs	The archiver. <code>r</code> = insert/replace files in the archive, <code>c</code> = create the archive if it doesn't exist (suppresses the warning if it does), <code>s</code> = write an index of symbols, which is what lets the linker later resolve function names without scanning every object file linearly.
SRCS	43 filenames	The complete, explicit list of source files that make up the mandatory library.
OBJS	\$(SRCS:.c=.o)	A substitution reference — Make's built-in text-replace syntax. It takes the <code>SRCS</code> list and swaps every <code>.c</code> suffix for <code>.o</code> , without maintaining a second hand-written list that could silently drift out of sync with <code>SRCS</code> .

2.3 — The Pipeline: .c → .o → .a



The pattern rule that drives this is:

```
%o: %.c libft.h
    $(CC) $(CFLAGS) -c $< -o $@
```

`%` is Make's wildcard — it means "for any stem, a file `stem.o` depends on `stem.c` and `libft.h`." `$<` expands to the first prerequisite (the `.c` file), `$@` expands to the target itself (the `.o` file). Listing `libft.h` as a prerequisite is deliberate: if you edit the header — say, changing a function's signature — every `.o` file depending on it is now considered stale and will be recompiled on the next `make`, even though none of the `.c` files themselves changed.

2.4 — Relinking: The Rule That Catches Careless Students

42's Common Instructions state plainly: *"your Makefile must not perform unnecessary relinking."* Here's precisely what that means and how this architecture satisfies it.

A naive Makefile might rebuild *everything* on every invocation — recompiling all 43 files and re-archiving them, even if only one file changed, or even if nothing changed at all. That's the failure mode "unnecessary relinking" refers to: wasted work performed because the dependency graph wasn't expressed precisely enough for `make` to know what's actually stale.

Scenario: you edit `ft_split.c`, then run `make`

```
ft_isalpha.o  (unchanged) ┘
ft_strlen.o   (unchanged) ┘
ft_split.o    (STALE)  ────┐─── only ft_split.c is recompiled
:              |           | the other 42 .o files are
ft_lstmap.o   (unchanged) ┘ left untouched on disk
```



```
ar rcs libft.a *.o
(re-archives – cheap, not a recompile)
```

MAKE COMPARES MTIMES PER-FILE. ONE STALE .O DOES NOT STALE THE OTHER 42.

This Makefile achieves correct behavior through two structural choices:

1. **Per-file pattern rule.** Because `%.o: %.c libft.h` expresses the dependency at the level of individual files rather than "rebuild everything," `make`'s built-in timestamp comparison does the filtering automatically. Touching one `.c` file marks exactly one `.o` file stale.
2. **\$(NAME) itself depends on \$(OBSJ), not on a phony always-run recipe.** `ar rcs libft.a $(OBSJ)` only executes if at least one `.o` in the list is newer than `libft.a` — so if you run `make` twice in a row with no edits in between, the second run prints nothing and does nothing. That's the textbook definition of "no unnecessary relinking."

Verifying it yourself: run `make`, then immediately run `make` again. A correctly structured Makefile produces zero output the second time — every target is already up to date. If it recompiles anything on that second run, the dependency graph is broken somewhere.

The `bonus:` target follows the identical logic — it currently reads `bonus: all`, so invoking it triggers the exact same up-to-date check against `$(OBSJ)` rather than an unconditional rebuild.

PART 3 • THE LIBRARY DEEP-DIVE

43 Functions, One Discipline

Every function below is graded on the same three axes: what it *does* (Technical Logic), why it's *shaped this way* under 42's Norm (Norminette & C Standards), and how it *survives contact with bad input* (Memory

Security). Functions are grouped by the concept they teach, not alphabetically — the order tells its own story.

3.1 — Character Classification & Case Conversion

Seven functions, one shape: take an `int c`, test it against a numeric range, return `1` / `0` or a transformed character. This is the simplest category in the library, and deliberately so — it's where 42 checks that you understand ASCII as *numbers*, not as opaque symbols.

ft_isalpha

CONCEPT: ASCII RANGE TESTING

```
int    ft_isalpha(int c)
{
    if ((c ≥ 'a' && c ≤ 'z') || (c ≥ 'A' && c ≤ 'Z'))
        return (1);
    return (0);
}
```

TECHNICAL LOGIC

ASCII places lowercase letters at codes 97–122 and uppercase at 65–90. The condition is two independent range checks joined by `||` — the function never needs to know *which* case matched, only that `c` falls in either band. Because character literals like `'a'` are themselves `int` constants in C, this comparison is pure integer arithmetic; no character-table lookup occurs.

NORMINETTE & C STANDARDS

Single `if`, two explicit `return` statements, no ternary (forbidden by Norm), no early-exit chains. This "guard, then fall through to a final return" shape is the canonical Norm-compliant pattern for a boolean-style function — it reads top-to-bottom with exactly one decision point.

MEMORY SECURITY

Stateless and read-only — takes a value, not a pointer, so there is nothing to leak and nothing to dereference. The parameter is `int`, not `char`, precisely so that `EOF` (-1) or any out-of-range integer can be passed safely without triggering the signed-`char` promotion bugs that plague naive reimplementations.

ft_isdigit

CONCEPT: ASCII RANGE TESTING

```
int ft_isdigit(int c)
{
    if (c ≥ '0' && c ≤ '9')
        return (1);
    return (0);
}
```

TECHNICAL LOGIC

Identical shape to `ft_isalpha`, testing the single contiguous ASCII band 48–57. This contiguity is what allows `c - '0'` to convert a digit character to its numeric value elsewhere in the library (notably inside `ft_atoi`) — the range check here is effectively a bounds guard for that conversion.

NORMINETTE & C STANDARDS

Same guard-then-return structure. Norm rewards this repetition of a known-safe pattern rather than "clever" one-liners — predictability is the point of a style standard.

MEMORY SECURITY

No pointers, no allocation, no failure mode. The full input domain of `int` is handled correctly, including negative values and values above 255.

ft_isalnum

CONCEPT: FUNCTION COMPOSITION

```
int    ft_isalnum(int c)
{
    if (ft_isalpha(c) || ft_isdigit(c))
        return (1);
    return (0);
}
```

TECHNICAL LOGIC

This is the first function in the library that **composes** rather than reimplements. Instead of writing three separate range checks (lower, upper, digit), it delegates to the two functions that already encode those ranges. This is a direct, deliberate application of the subject's own hint: "some of the functions from Part 1 may be useful for implementing the functions below."

NORMINETTE & C STANDARDS

Composition over duplication is not just cleaner — it's a Norm-adjacent virtue, since every duplicated range check is a second place a future bug (or a future edit) can diverge from the first. One source of truth per concept.

MEMORY SECURITY

Inherits the safety properties of both dependencies. No new failure surface is introduced by composition.

ft_isascii

CONCEPT: ASCII RANGE TESTING

```
int    ft_isascii(int c)
{
    if (c ≥ 0 && c ≤ 127)
        return (1);
    return (0);
}
```

TECHNICAL LOGIC

The full 7-bit ASCII table spans 0–127 inclusive — this is the widest range of any classification function in the library, deliberately including control characters (0–31) and `DEL` (127), which `ft_isprint` explicitly excludes. Conflating these two functions is the single most common student bug in this part of the subject.

NORMINETTE & C STANDARDS

Identical structural template to its siblings — the value of a style standard is exactly this: once you've internalized the shape, every function in the category reads interchangeably.

MEMORY SECURITY

Correctly returns `0` for negative input and for anything ≥ 128 , rather than relying on undefined or implementation-specific char-signedness behavior.

ft_isprint

CONCEPT: ASCII RANGE TESTING

```
int ft_isprint(int c)
{
    if (c ≥ 32 && c ≤ 126)
        return (1);
    return (0);
}
```

TECHNICAL LOGIC

32 is the space character; 126 is `~` — the narrowest, "displayable ink on a page" band of ASCII. Everything below 32 is a control character (tab, newline, escape...), and 127 (`DEL`) is explicitly excluded, which is why this range's upper bound is 126, not 127 like `ft_isascii`.

NORMINETTE & C STANDARDS

The consistent template across all five classification functions is itself a demonstration of Norm discipline — a reviewer can verify all five correctly at a glance because they share one silhouette.

MEMORY SECURITY

No pointer dereferencing; behavior is fully defined across the entire `int` domain.

ft_toupper / ft_tolower

CONCEPT: BITWISE DISTANCE IN ASCII

```
int ft_toupper(int c)
{
    if (c ≥ 'a' && c ≤ 'z')
        return (c - 32);
    return (c);
}

int ft_tolower(int c)
{
    if (c ≥ 'A' && c ≤ 'Z')
        return (c + 32);
    return (c);
}
```

TECHNICAL LOGIC

The magic number 32 is not arbitrary: in the ASCII table, every uppercase letter sits exactly 32 codepoints below its lowercase counterpart (`'A'` =65, `'a'` =97). Because this offset is *constant across the entire alphabet*, one subtraction converts any lowercase letter to uppercase, and one addition reverses it — no lookup table needed. Non-letter input is returned completely unmodified, which matters because callers (like `ft_strmapi` callback examples) may apply these functions to punctuation or digits without checking first.

NORMINETTE & C STANDARDS

The guard-and-fallthrough shape persists even here, where the "fallthrough" case is an identity transform rather than a boolean default — consistency of pattern over cleverness of implementation.

MEMORY SECURITY

Pure value transforms, no aliasing risk, no allocation. Safe against every possible `int` input including negative values and values from `EOF`.

3.2 — Memory Primitives

This is where the library stops treating memory as "a string" and starts treating it as *bytes*. Every function below operates through `unsigned char *` — the only pointer type in C guaranteed to let you inspect memory one raw byte at a time, regardless of what type actually lives there.

ft_memset

CONCEPT: BYTE-LEVEL ITERATION

```
void    *ft_memset(void *b, int c, size_t len)
{
    unsigned char    *ptr;
    size_t            i;

    ptr = (unsigned char *)b;
    i = 0;
    while (i < len)
    {
        ptr[i] = (unsigned char)c;
        i++;
    }
    return (b);
}
```

TECHNICAL LOGIC

The incoming `void *b` carries no size information for indexing — you cannot write `b[i]` on a `void *`. The cast to `unsigned char *` gives the compiler a concrete, one-byte stride, so `ptr[i]` advances exactly one byte per iteration. `c` is truncated to `unsigned char` before the write — this matches the real `memset`'s documented behavior of only using the low 8 bits of its `int` argument, even if a caller passes something larger.

NORMINETTE & C STANDARDS

Two declarations at the top of the block, separated by a blank line from the executable statements — the mandatory Norm layout. The function returns `b` (the original, untouched pointer) rather than `ptr`, matching the real `memset` contract exactly, even though internally the function works through `ptr`.

MEMORY SECURITY

Writes exactly `len` bytes — never more. If `len` exceeds the actual allocated size of `b`, that's a caller error (buffer overflow) no reimplementations can protect against without the size being passed correctly in the first place; this mirrors real `memset`'s contract precisely.

ft_bzero

CONCEPT: BYTE-LEVEL ITERATION

```
void    ft_bzero(void *s, size_t n)
{
    unsigned char    *ptr;
    size_t            i;

    ptr = (unsigned char *)s;
    i = 0;
    while (i < n)
    {
        ptr[i] = 0;
        i++;
    }
}
```

TECHNICAL LOGIC

Structurally identical to `ft_memset` with the fill byte fixed at `0` and no return value — `bzero` was historically a BSD convenience wrapper for exactly this common case. This implementation is reused internally by `ft_calloc` to zero freshly allocated memory.

NORMINETTE & C STANDARDS

Consistent byte-iteration template shared with `ft_memset` — two functions that look nearly identical because they *are* nearly identical in purpose, which is exactly what a reader should expect.

MEMORY SECURITY

Same contract as `ft_memset`: writes exactly `n` bytes, no more. Used correctly by `ft_calloc` with a pre-overflow-checked size (see 3.4).

ft_memcpy

CONCEPT: NON-OVERLAPPING BYTE COPY

```

void    *ft_memcpy(void *dst, const void *src, size_t n)
{
    unsigned char    *d;
    const unsigned char    *s;
    size_t            i;

    i = 0;
    d = (unsigned char *)dst;
    s = (const unsigned char *)src;
    while (i < n)
    {
        d[i] = s[i];
        i++;
    }
    return (dst);
}

```

```

src:  [A][B][C][D][E]
      | | | | |
      ▼ ▼ ▼ ▼ ▼
dst:  [.] [.] [.] [.] [.] → [A][B][C][D][E]

```

LEFT-TO-RIGHT, ONE BYTE AT A TIME. SAFE ONLY WHEN SRC AND DST DO NOT

TECHNICAL LOGIC

Two typed pointers are derived from the two `void *` parameters — note `s` is declared `const unsigned char *`, preserving the read-only guarantee the caller made about `src`. The loop is a straightforward forward copy. Unlike `ft_memmove`, this function performs **no overlap detection** — that's not a bug, it's the documented contract of `memcpy` itself: behavior is undefined if the regions overlap, and the caller is responsible for choosing `memmove` instead when overlap is possible.

NORMINETTE & C STANDARDS

Historical note on this exact function: an earlier revision of this file declared the second parameter as `const char *src` instead of `const void *src` — a signature mismatch against the header that the compiler caught immediately as `error: conflicting types for 'ft_memcpy'`. This is a useful case study in why `-Werror` exists: the fix was a single word, but the header contract makes such mismatches impossible to ship silently.

MEMORY SECURITY

Copies exactly `n` bytes. The single documented caveat — overlap — is explicitly the reason `ft_memmove` exists as a separate, slower-but-safer function. Calling `ft_memcpy` on overlapping regions is undefined behavior by design, matching the real libc function precisely.

ft_memmove

CONCEPT: DIRECTION-AWARE COPYING

```

void    *ft_memmove(void *dst, const void *src, size_t len)
{
    unsigned char    *d;
    const unsigned char    *s;

    if (!dst && !src)
        return (NULL);
    d = (unsigned char *)dst;
    s = (const unsigned char *)src;
    if (d > s)
    {
        while (len > 0)
        {
            len--;
            d[len] = s[len];
        }
    }
    else
    {
        while (len--)
            *d++ = *s++;
    }
    return (dst);
}

```

Case: dst is AHEAD of src ($d > s$) – copy BACKWARD, high index first

```

src: [A][B][C][D][E]
dst:  [.] [.] [.] [.] [.]
      ▲                               (copying index 4 first prevents it being
      └─ last write overwritten before it's read)

```

Case: dst is BEHIND src ($d \leq s$) – copy FORWARD, low index first

```

src:  [A][B][C][D][E]
dst: [.] [.] [.] [.] [.]
      ▲                               (copying index 0 first is safe – src[0]
      └─ first write is never touched by an earlier dst write)

```

THE DIRECTION OF TRAVERSAL IS CHOSEN SO NO BYTE IS OVERWRITTEN BEFO

TECHNICAL LOGIC

This is the most conceptually important function in the memory-primitives group. When `dst` and `src` overlap and `dst` sits at a *higher* address, a naive forward copy would overwrite `src[k]` with `src[k - offset]` before the original `src[k]` has been read — corrupting the tail of the copy. The fix is to detect this case ($d > s$) and copy **backward**,

from the last byte to the first, so every byte is read before the write that could clobber it ever happens. When `d ≤ s`, a forward copy is always safe regardless of overlap, so the function takes the simpler, faster pointer-increment path (`*d++ = *s++`).

NORMINETTE & C STANDARDS

The `if (!dst && !src) return (NULL)` guard is a defensive addition beyond the strict letter of the man page (real `memmove` has undefined behavior on null pointers regardless of `len`) — a reasonable, Norm-legal judgment call that costs one branch and avoids a crash in the specific double-null case.

MEMORY SECURITY

This function is precisely what makes safe self-overlapping shifts possible — for example, shifting an array's contents left or right in place. Getting the direction check backward (using `≥` instead of `>`, or inverting the branches) is the single most common way students silently corrupt data in this function while it still "compiles fine" — there's no compiler warning for a logically reversed overlap check.

3.3 — String Inspection & Comparison

Eight functions that read strings without modifying them — searching, comparing, and converting. This is the category most dependent on getting loop *termination conditions* exactly right, since an off-by-one here means reading past a buffer's end.

ft_strlen

CONCEPT: SENTINEL-TERMINATED TRAVERSAL

```
size_t    ft_strlen(const char *s)
{
    size_t    len;

    len = 0;
    while (s[len] != '\0')
    {
        len++;
    }
    return (len);
}
```

TECHNICAL LOGIC

C strings carry no length field — length is determined entirely by scanning forward until the null terminator `'\0'` (byte value 0) is found. This function is the foundation every other string function in the library is built on top of, directly or indirectly (via `ft_strlcpy`).

NORMINETTE & C STANDARDS

Return type is `size_t` — an unsigned type — matching the real `strlen` exactly, which matters because it means length can never be negative, but also means subtracting two lengths carelessly can silently wrap around to a huge positive number (a trap several other functions, like `ft_strlcat`, must guard against explicitly).

MEMORY SECURITY

Relies entirely on the caller passing a properly null-terminated buffer — there's no way for any string function to verify this from inside; passing a non-terminated `char *` is undefined behavior in C generally, not a libft-specific limitation.

ft_strlcpy / ft_strlcat

CONCEPT: BOUNDED WRITES

```

size_t    ft_strlcpy(char *dst, const char *src, size_t dstsize)
{
    size_t    src_len;
    size_t    i;

    src_len = 0;
    while (src[src_len] != '\0')
        src_len++;
    if (dstsize == 0)
        return (src_len);
    i = 0;
    while (src[i] != '\0' && i < (dstsize - 1))
    {
        dst[i] = src[i];
        i++;
    }
    dst[i] = '\0';
    return (src_len);
}

```

TECHNICAL LOGIC

The entire point of the "l" in `strlcpy` ("length-bounded") is that it **never writes past** `dstsize`, unlike the notoriously unsafe `strcpy`. The copy loop stops at `dstsize - 1` to always reserve one byte for the terminating `'\0'`. Critically, the **return value is the full source length**, not the number of bytes actually copied — this lets a caller detect truncation by checking `if (ft_strlcpy(...) ≥ dstsize)`.

`ft_strlcat` follows the same discipline in its own file: it first measures how much of `dst` is already used (bounded by `dstsize`), and — this is the detail most reimplementations miss — if `dst` is *already* at or past its capacity (`dst_len ≥ dstsize`), it returns immediately without touching `dst` at all, rather than attempting a partial, corrupting write.

NORMINETTE & C STANDARDS

`dstsize == 0` is checked as an explicit early branch before any indexing occurs — necessary because with `dstsize` as an unsigned `size_t`, computing `dstsize - 1` when `dstsize` is already `0` would wrap around to `SIZE_MAX`, turning a bounded loop into an unbounded one.

MEMORY SECURITY

This pairing (`strlcpy` / `strlcat`) is the primary defense the rest of the library relies on for building strings without overflow — `ft_strjoin`, `ft_strtrim`, and `ft_substr` all

delegate their actual byte-copying to `ft_strlcpy` rather than reimplementing bounds logic each time.

ft_strchr / ft_strrchr

CONCEPT: FORWARD VS. REVERSE SEARCH

```

char    *ft_strchr(const char *s, int c)
{
    size_t    i;

    i = 0;
    while (s[i] != '\0')
    {
        if (s[i] == (char)c)
            return ((char *)&s[i]);
        i++;
    }
    if ((char)c == '\0')
        return ((char *)&s[i]);
    return (NULL);
}

```

```

s: h e l l o \0
    |               |
    |← strchr —     | scans LEFT → RIGHT, stops at first match
    |               |
    |—— strrchr →    | scans full length first, then RIGHT → LEFT

```

SAME SIGNATURE, OPPOSITE TRAVERSAL DIRECTION — HENCE 'R' FOR REVERSE

TECHNICAL LOGIC

The subtle, easy-to-miss detail in both functions: **the null terminator itself is a valid search target**. Real `strchr(s, '\0')` is defined to return a pointer to the terminating null byte, not `NULL` — this implementation explicitly checks for that case *after* the main loop exits, rather than treating loop-exit as automatic failure.

`ft_strrchr` takes a different approach: it first walks forward to find the string's length (landing exactly on the null terminator), then walks *backward* from there. Because the initial position *is* the null terminator, the very first check inside the descending loop naturally also handles the `c == '\0'` case, without a separate branch.

NORMINETTE & C STANDARDS

Both cast `c` to `char` before comparison — necessary because `c` arrives as `int` (matching the real prototype, which allows passing values like `EOF` without truncation warnings at the call site), but string bytes are compared as `char`.

MEMORY SECURITY

Both return a pointer *into* the original string, never an allocation — callers must not `free()` the result, exactly matching real libc behavior. No bounds are exceeded because the loop conditions are anchored to the null terminator itself.

ft_strncmp / ft_memcmp

CONCEPT: BYTE-DIFFERENCE RETURN CONVENTION

```
int ft_strncmp(const char *s1, const char *s2, size_t n)
{
    size_t i;

    i = 0;
    if (n == 0)
        return (0);
    while (s1[i] != '\0' && s1[i] == s2[i] && i < (n - 1))
        i++;
    return ((unsigned char)s1[i] - (unsigned char)s2[i]);
}
```

TECHNICAL LOGIC

Both functions share the same return-value contract: not just "equal or not," but the actual *signed numeric difference* between the first mismatching byte pair — a negative result means `s1` sorts before `s2`, positive means after, zero means the compared region is identical. The loop condition in `ft_strncmp` stops early on three independent conditions: hitting the end of `s1`, finding a mismatch, or exhausting `n - 1` comparisons — after which the final line computes the difference at whichever position the loop stopped, whether that's a real mismatch or the natural end of the string.

NORMINETTE & C STANDARDS

The `n == 0` guard exists for the same reason as in `ft_strlcpy`: with `n` unsigned, an unguarded `n - 1` would wrap to `SIZE_MAX`. Both bytes are cast to `unsigned char` before subtraction — required because plain `char` may be signed on a given platform, which would make the sign of the returned difference platform-dependent for byte values above 127.

MEMORY SECURITY

Read-only, bounded strictly by `n` — never reads past the shorter of the two strings or past `n` bytes, whichever comes first.

ft_memchr

CONCEPT: BYTE-LEVEL SEARCH

```
void    *ft_memchr(const void *s, int c, size_t n)
{
    size_t      i;
    const unsigned char  *ptr;

    ptr = (const unsigned char *)s;
    i = 0;
    while (i < n)
    {
        if (ptr[i] == (unsigned char)c)
            return ((void *)&ptr[i]);

        i++;
    }
    return (NULL);
}
```

TECHNICAL LOGIC

Unlike `ft_strchr`, this function has no concept of a null terminator stopping the search — it operates purely on `n` raw bytes, making it usable on binary data that may legitimately contain zero bytes in the middle. This is the defining difference between the "mem" and "str" families throughout the library.

NORMINETTE & C STANDARDS

Returns `void *`, not `char *` — matching the real signature, since the function is defined to operate on arbitrary memory, not specifically text.

MEMORY SECURITY

Bounded strictly by `n`; will never read past the caller-specified length regardless of what bytes actually live in memory.

ft_strnstr

CONCEPT: NESTED BOUNDED SEARCH

```

char    *ft_strnstr(const char *haystack, const char *needle, size_t len)
{
    size_t    i;
    size_t    j;

    i = 0;
    if (needle[0] == '\0')
        return ((char *)haystack);
    while (haystack[i] != '\0' && i < len)
    {
        j = 0;
        while (haystack[i + j] == needle[j] && (i + j) < len)
        {
            if (needle[j + 1] == '\0')
                return ((char *)&haystack[i]);
            j++;
        }
        i++;
    }
    return (NULL);
}

```

TECHNICAL LOGIC

A nested-loop substring search: the outer loop `i` tries each starting position in `haystack`; the inner loop `j` checks whether `needle` matches starting there. The match is confirmed the instant `needle[j + 1] == '\0'` — meaning every character of `needle` matched up to and including its last — rather than waiting to separately verify needle's length first.

NORMINETTE & C STANDARDS

The empty-needle case (`needle[0] == '\0'`) is handled as an explicit early return matching the documented real behavior: an empty needle is defined to match at the very start of haystack, immediately.

MEMORY SECURITY

Both loop conditions independently enforce the `len` bound (`i < len` and `(i + j) < len`), so the function can never read past position `len` in `haystack` even if `needle` is long and `haystack` lacks a null terminator within that range.

ft_atoi

CONCEPT: MANUAL LEXICAL PARSING

```

int ft_atoi(const char *str)
{
    size_t i;
    int sign;
    int result;

    i = 0;
    sign = 1;
    result = 0;
    while (str[i] == ' ' || (str[i] ≥ 9 && str[i] ≤ 13))
        i++;
    if (str[i] == '-' || str[i] == '+')
    {
        if (str[i] == '-')
            sign = -1;
        i++;
    }
    while (str[i] ≥ '0' && str[i] ≤ '9')
    {
        result = (result * 10) + (str[i] - '0');
        i++;
    }
    return (result * sign);
}

```

```

" -42abc"
▲▲▲▲▲▲▲
|||||└─ digits consumed: '4','2' → result builds 4, then 42
|||||
|||||└─ sign consumed: '-' → sign = -1
|└─── whitespace skipped (space + \t\n\v\f\r, codes 9-13)
└─── parsing stops the instant a non-digit ('a') appears

```

THREE SEQUENTIAL PHASES: SKIP WHITESPACE → READ OPTIONAL SIGN → ACCUMULATE DIGITS

TECHNICAL LOGIC

Three sequential, non-overlapping phases, matching the real `atoi`'s documented grammar exactly. Phase one skips any mix of spaces and the whitespace control codes 9 through 13 (tab, newline, vertical tab, form feed, carriage return). Phase two consumes at most one `+` or `-`. Phase three accumulates digits via the classic `result = result * 10 + digit` pattern, which is simply long multiplication performed one digit at a time — and simply *stops*, without error, the moment a non-digit character appears (including a second sign, or letters).

NORMINETTE & C STANDARDS

Three separate, purpose-named variables (`i` , `sign` , `result`) declared together at the top, each with a single clear responsibility — exactly the kind of variable hygiene Norm is designed to enforce, since a reviewer can understand the function's state just from the declaration block before reading a single line of logic.

MEMORY SECURITY

Edge case worth naming explicitly: `INT_MIN` . For the input `"-2147483648"` , digits accumulate as a *positive* `int` up to `2147483648` before the final multiplication by `sign` — but `2147483648` itself overflows a 32-bit signed `int` (max `2147483647`) one step before the sign is even applied. This is technically undefined behavior per the C standard for a strictly conforming reimplementation, but it is the exact same behavior the real glibc `atoi` exhibits on this input in practice (the overflow wraps consistently to `INT_MIN` on twos-complement hardware), so this implementation's behavior matches the reference it's being graded against.

3.4 — Heap Allocation

The first two functions in the library that call `malloc` directly. Everything from here on must be paired with a corresponding `free()` by the caller — this is the boundary where memory-leak discipline starts to matter.

ft_calloc

CONCEPT: OVERFLOW-CHECKED MULTIPLICATION

```

void    *ft_calloc(size_t count, size_t size)
{
    void    *ptr;
    size_t  total_size;

    if (count != 0 && size > (size_t)-1 / count)
        return (NULL);
    total_size = count * size;
    ptr = malloc(total_size);
    if (ptr == NULL)
        return (NULL);
    ft_bzero(ptr, total_size);
    return (ptr);
}

```

TECHNICAL LOGIC

The entire reason `calloc` exists as a separate function from `malloc` + manual zeroing is this exact overflow check. If `count * size` were computed directly and it overflowed `size_t`, the wrapped-around result could be a *small* number — `malloc` would happily succeed with a tiny buffer, and the caller, believing they have `count * size` usable bytes, would write far past its end. The guard rearranges the multiplication into a division (`size > SIZE_MAX / count`) specifically so the overflow check itself cannot overflow — `(size_t)-1` is a standard, portable way to obtain `SIZE_MAX` without needing to `#include <limits.h>`.

NORMINETTE & C STANDARDS

Reuses `ft_bzero` rather than a hand-rolled zeroing loop — consistent with the library's composition-over-duplication discipline established back in `ft_isalnum`.

MEMORY SECURITY

Handles the documented edge case explicitly: when `count` or `size` is `0`, the overflow guard is skipped (`count != 0` short-circuits), `total_size` becomes `0`, and `malloc(0)` is called — which on this platform returns a unique, non-NULL pointer that is safely passable to `free()`, exactly matching the behavior the subject explicitly mandates.

ft_strdup

CONCEPT: MEASURE-THEN-ALLOCATE

```
char    *ft_strdup(const char *s1)
{
    char    *dest;
    size_t   len;

    len = 0;
    while (s1[len] != '\0')
        len++;
    dest = (char *)malloc(sizeof(char) * (len + 1));
    if (dest == NULL)
        return (NULL);
    ft_strlcpy(dest, s1, len + 1);
    return (dest);
}
```

TECHNICAL LOGIC

The canonical two-phase allocation pattern used throughout the rest of the library: first **measure** exactly how much space is needed (here, a manual length scan identical in shape to `ft_strlen`), then **allocate exactly that much** — `len + 1` to include room for the terminator — and finally copy. Delegating the actual copy to `ft_strlcpy` means the bounds-safety logic lives in exactly one place in the entire library.

NORMINETTE & C STANDARDS

`sizeof(char)` is technically always `1` by the C standard's own guarantee, making it redundant multiplicatively — but it's included for self-documentation, a defensible and common style choice that costs nothing at runtime.

MEMORY SECURITY

The `malloc` failure path is checked before any write occurs, and `NULL` is propagated rather than dereferenced — matching the subject's explicit contract that this function returns `NULL` if the allocation fails. The caller is responsible for eventually calling `free()` on the returned pointer; this function has no way to leak memory itself, only to correctly hand ownership to its caller.

3.5 — String Construction

Five functions that build *new* strings on the heap from existing ones. This is the category where edge-case handling around bounds — start indices past the string's end, lengths that overshoot — separates a technically-compiling function from a genuinely correct one.

ft_substr

CONCEPT: CLAMPED RANGE EXTRACTION

```

char    *ft_substr(char const *s, unsigned int start, size_t len)
{
    char    *sub;
    size_t   s_len;

    if (!s)
        return (NULL);
    s_len = ft_strlen(s);
    if (start ≥ s_len)
        return (ft_strdup(""));
    if (len > s_len - start)
        len = s_len - start;
    sub = (char *)malloc(sizeof(char) * (len + 1));
    if (!sub)
        return (NULL);
    ft_strlcpy(sub, s + start, len + 1);
    return (sub);
}

```

```
s = "hello world"   start=6   len=100 (deliberately too large)
```

```

index:  0    5 6          10
        h e l l o _ w o r l d \0

```

```

        └──────────┘
        actual available = s_len - start = 5
        len gets CLAMPED from 100 down to 5

```

```
result → "world" (never reads past s's actual end)
```

TECHNICAL LOGIC

Two defensive clamps, in order. First: if `start` is already beyond the string's end, there is nothing to extract — return an empty (but valid, allocated) string rather than attempting arithmetic that would underflow. Second: even when `start` is valid, the caller-requested `len` may ask for more characters than actually remain — `len` is silently reduced to `s_len - start`, the true remaining length, before any allocation happens.

NORMINETTE & C STANDARDS

The clamping logic is expressed as two independent, sequential `if` guards rather than one compound condition — each guard protects exactly one downstream computation (the first protects the subtraction on the next line; the second protects the allocation size), which keeps each condition readable in isolation.

MEMORY SECURITY

This is the exact function that, in an earlier revision of this project, was accidentally replaced by a copy-pasted body of `ft_strdup` — a bug invisible to both the compiler and Norminette, since the swapped-in code was itself valid, norm-compliant C. It only surfaced when a real program tried to link against `ft_substr` and the linker had no symbol by that name. The lesson generalizes: **a clean compile and a clean Norm check are necessary but not sufficient evidence of correctness** — only running the function proves it does what its name claims.

ft_strjoin

CONCEPT: TWO-PHASE CONCATENATION

```
char    *ft_strjoin(char const *s1, char const *s2)
{
    char    *joined;
    size_t   s1_len;
    size_t   s2_len;

    if (!s1 || !s2)
        return (NULL);
    s1_len = 0;
    while (s1[s1_len] != '\0')
        s1_len++;
    s2_len = 0;
    while (s2[s2_len] != '\0')
        s2_len++;
    joined = (char *)malloc(sizeof(char) * (s1_len + s2_len + 1));
    if (joined == NULL)
        return (NULL);
    ft_strlcpy(joined, s1, s1_len + 1);
    ft_strlcat(joined, s2, s1_len + s2_len + 1);
    return (joined);
}
```

TECHNICAL LOGIC

Measure both input lengths first, allocate exactly `s1_len + s2_len + 1` bytes in one call, then use `ft_strlcpy` to place `s1` and `ft_strlcat` to append `s2` — note the third argument to `ft_strlcat` is the *total* buffer size (`s1_len + s2_len + 1`), not just `s2`'s length, since `strlcat`'s contract requires the full destination capacity to correctly bound its internal scan.

NORMINETTE & C STANDARDS

A single allocation sized precisely, rather than an oversized buffer with slack — this matters both for Norm's implicit expectation of deliberate, justified sizing and for memory efficiency at scale.

MEMORY SECURITY

`NULL`-checks both inputs before any length computation — calling `ft_strlen`-style scanning on a `NULL` pointer would itself be undefined behavior, so the guard must come first, not after.

MEMORY SECURITY

Handles the "entire string is trim characters" edge case correctly: `start` and `end` converge to the same index, the allocation size becomes exactly `1` (room for just `'\0'`), and the result is a valid, empty, freeable string — not a crash.

ft_split

CONCEPT: TWO-PASS TOKENIZATION

```
static size_t    count_words(char const *s, char c)
{
    size_t    count;
    size_t    i;

    count = 0;
    i = 0;
    while (s[i] != '\0')
    {
        if (s[i] != c && (s[i + 1] == c || s[i + 1] == '\0'))
            count++;
        i++;
    }
    return (count);
}

static char      *alloc_word(char const *s, char c, size_t *index)
{
    size_t    len;
    size_t    i;
    char      *word;

    len = 0;
    while (s[*index] == c && s[*index] != '\0')
        (*index)++;
    while (s[*index + len] != c && s[*index + len] != '\0')
        len++;
    word = (char *)malloc(sizeof(char) * (len + 1));
    if (word == NULL)
        return (NULL);
    i = 0;
    while (i < len)
    {
        word[i] = s[*index];
        (*index)++;
        i++;
    }
    word[i] = '\0';
    return (word);
}

char      **ft_split(char const *s, char c)
{
    char      **result;
    size_t    words;
    size_t    i;
    size_t    s_idx;

    if (s == NULL)
        return (NULL);
    words = count_words(s, c);
    result = (char **)malloc(sizeof(char *) * (words + 1));
    if (result == NULL)
        return (NULL);
    i = 0;
    s_idx = 0;
```

```

    while (i < words)
    {
        result[i] = alloc_word(s, c, &s_idx);
        if (result[i] == NULL)
            return (NULL);
        i++;
    }
    result[i] = NULL;
    return (result);
}

```

```
s = "a,,b,c"    delimiter = ','
```

pass 1 (count_words): scans once, counts transitions INTO a word
→ finds 3 words: "a" "b" "c" (empty gaps skipped)

pass 2 (per word): alloc_word() called 3 times, s_idx threaded
through by pointer so each call resumes
exactly where the previous one stopped

result:

"a"	"b"	"c"	NULL
-----	-----	-----	------

← NULL-terminated array

WORD COUNT MUST BE KNOWN BEFORE THE ARRAY IS SIZED — HENCE TWO PASS

TECHNICAL LOGIC

The most structurally complex function in the library, and the only one requiring `static` helper functions — mandated by the subject specifically to keep each function under the 25-line limit while breaking a genuinely multi-step algorithm into named, testable pieces. `count_words` performs a single forward scan, incrementing its counter only at the exact character that is *the last character of a word* — detected as "not the delimiter, AND the next character is either the delimiter or the end of string." This detects word boundaries without needing a "was I inside a word" state flag.

`alloc_word` receives `s_idx` as a `size_t *` — a pointer to the caller's index variable — so that each successive call to `alloc_word` resumes scanning exactly where the previous call left off, rather than re-scanning from the start each time. It first skips any run of delimiter characters, then measures the word length, then allocates and copies.

NORMINETTE & C STANDARDS

Two `static` functions confine their scope to this translation unit only, exactly as the subject instructs: *"define them as static functions to restrict their scope to the appropriate file."* Splitting the algorithm this way keeps every single function — including the public `ft_split` itself — comfortably under 25 lines.

MEMORY SECURITY

The partial-failure path deserves scrutiny: if `alloc_word` fails on, say, the third of five words, `result[0]` and `result[1]` were already successfully allocated but the function returns `NULL` without freeing them — a genuine, narrow memory leak that only triggers under simultaneous system-level allocation failure. This is a defensible simplification for a student project (the subject only requires returning `NULL` "if any allocation fails," not a full rollback), but a production-grade version would track and free `result[0..i-1]` and the array itself before returning `NULL`. Worth knowing the gap exists even if you choose not to close it.

ft_itoa

CONCEPT: REVERSE DIGIT EXTRACTION

```
static size_t get_num_len(int n)
{
    size_t len;

    len = 0;
    if (n ≤ 0)
        len++;
    while (n ≠ 0)
    {
        n /= 10;
        len++;
    }
    return (len);
}

char *ft_itoa(int n)
{
    char *str;
    size_t len;
    long int num;

    num = n;
    len = get_num_len(n);
    str = (char *)malloc(sizeof(char) * (len + 1));
    if (str == NULL)
        return (NULL);
    str[len] = '\0';
    if (num == 0)
        str[0] = '0';
    if (num < 0)
    {
        str[0] = '-';
        num = -num;
    }
    while (num > 0)
    {
        len--;
        str[len] = (num % 10) + '0';
        num /= 10;
    }
    return (str);
}
```

```

n = -42          num promoted to `long` FIRST → -42L

get_num_len(-42): n ≤ 0 → len starts at 1 (for the '-')
                  -42 → -4 → 0 : two divisions → len becomes 3
                  total len = 3   ("-42" is 3 characters)

build phase (num negated to +42, written back-to-front):
index:  0   1   2
str:    [-] [4] [2] \0
      ▲   ▲   ▲
      |   |   |
      |   |   └─ written FIRST (num%10=2, num→4)
      |   └─── written SECOND (num%10=4, num→0)
      └────── sign written directly, untouched by the loop

```

TECHNICAL LOGIC

The classic "digits come out backward" problem: `num % 10` always yields the *least significant* digit first, so the write index must start at the *end* of the buffer and move toward the front, one `len--` per digit. `get_num_len` mirrors this exactly by counting divisions by 10 until `n` reaches zero, plus one extra slot reserved up front if the number is zero or negative (for the digit `'0'` itself, or for the `'-'` sign).

NORMINETTE & C STANDARDS

A dedicated `static` helper for length calculation, matching the same "measure then allocate" two-phase discipline seen in `ft_strdup` and `ft_substr` — a pattern that recurs throughout this library specifically because Norm's declarations-at-top rule makes interleaving "figure out the size" logic with "write the bytes" logic awkward inside a single function.

MEMORY SECURITY

The defining edge case of this function is `INT_MIN` (`-2147483648`). Negating `INT_MIN` as a plain `int` overflows, because the positive equivalent (`2147483648`) exceeds `INT_MAX` (`2147483647`) — undefined behavior in C. This implementation sidesteps the entire problem by immediately promoting `n` into `num`, a `long int`, *before* any negation happens. On any platform where `long` is wider than `int` (the standard case on 64-bit Linux, where `long` is 64 bits), `-num` when `num == INT_MIN` is computed entirely within `long`'s much larger range and never overflows. This single design decision — promote before negate — is what separates a correct `ft_itoa` from one that silently corrupts on exactly one specific input out of four billion.

3.6 — Function Pointers & File-Descriptor I/O

The library's most "meta" category: functions that take *other functions* as arguments, plus the small family of output primitives every one of them is ultimately built from.

ft_strmapi / ft_striteri

CONCEPT: CALLBACK-DRIVEN TRANSFORMATION

```
char    *ft_strmapi(char const *s, char (*f)(unsigned int, char))
{
    size_t    len;
    size_t    i;
    char      *str;

    if (!s || !f)
        return (NULL);
    len = ft_strlen(s);
    str = (char *)malloc(sizeof(char) * (len + 1));
    if (str == NULL)
        return (NULL);
    i = 0;
    while (i < len)
    {
        str[i] = f(i, s[i]);
        i++;
    }
    str[i] = '\0';
    return (str);
}
```

TECHNICAL LOGIC

`char (*f)(unsigned int, char)` declares `f` as a pointer to a function taking an index and a character, returning a transformed character — the classic C mechanism for letting a caller inject custom per-character logic (uppercasing, ROT13, digit-shifting) without `ft_strmapi` itself knowing or caring what the transformation is. The loop calls `f(i, s[i])` for every index, writing the result into a freshly allocated buffer — `s` itself is never mutated. `ft_striteri` is the in-place sibling: same calling convention, but `f` receives a *pointer* to the character (`char *`) so it can modify the original string directly, and the function returns nothing because there's no new buffer to hand back.

NORMINETTE & C STANDARDS

Function-pointer parameter syntax is inherently dense, but both declarations are still laid out on one line at the parameter-list level, matching the exact prototype text from the header — deviating from the header's declared spacing/parens here would be a signature mismatch, not a style choice.

MEMORY SECURITY

Both guard against `NULL` for *both* the string and the function pointer — calling through a `NULL` function pointer is undefined behavior (typically an immediate crash), so this

check is not optional defensiveness, it's the only thing standing between a bad call and a segfault.

ft_putchar_fd /
ft_putstr_fd /
ft_putendl_fd /
ft_putnbr_fd

CONCEPT: THE WRITE() SYSCALL BOUNDARY

```
void    ft_putchar_fd(char c, int fd)
{
    write(fd, &c, 1);
}

void    ft_putstr_fd(char *s, int fd)
{
    size_t    len;

    if (!s || fd < 0)
        return ;
    len = ft_strlen(s);
    write(fd, s, len);
}

void    ft_putendl_fd(char *s, int fd)
{
    if (!s)
        return ;
    ft_putstr_fd(s, fd);
    ft_putchar_fd('\n', fd);
}

void    ft_putnbr_fd(int n, int fd)
{
    long int    num;

    num = n;
    if (num < 0)
    {
        ft_putchar_fd('-', fd);
        num = -num;
    }
    if (num ≥ 10)
        ft_putnbr_fd(num / 10, fd);
    ft_putchar_fd((num % 10) + '0', fd);
}
```

```

ft_putnbr_fd(-123, 1)
|
|─ num < 0 → print '-', num becomes 123
|
|─ num ≥ 10 → RECURSE with num/10 = 12
|   |─ num ≥ 10 → RECURSE with num/10 = 1
|   |   |─ num < 10 → base case, print '1'
|   |   (unwind)
|   |─ AFTER recursive call returns, print '2'
|   (unwind)
|─ AFTER recursive call returns, print '3'

output order:  -  1  2  3

RECURSION HAPPENS BEFORE THE PRINT — THAT'S WHAT REVERSES %10'S NA

```

TECHNICAL LOGIC

This whole family sits directly on top of the single system call `write(fd, buffer, count)` — no buffering, no `printf`-style formatting machinery underneath. `ft_putstr_fd` writes a string in one syscall using its measured length. `ft_putendl_fd` composes two lower-level calls (`ft_putstr_fd` then a newline via `ft_putchar_fd`) rather than duplicating either. `ft_putnbr_fd` is the most interesting: it solves the exact same "digits come out backward" problem as `ft_itoa`, but through **recursion instead of a buffer** — the recursive call for `num / 10` happens *before* the current digit is printed, which means the most significant digit's print call is the deepest one on the call stack, and printing naturally happens in the correct left-to-right order as the stack unwinds.

NORMINETTE & C STANDARDS

The recursive structure keeps `ft_putnbr_fd` under the 25-line limit without needing a helper — recursion here isn't a stylistic flourish, it's the more Norm-compact solution compared to an iterative version that would need an auxiliary digit buffer.

MEMORY SECURITY

Same promote-to-`long`-before-negate technique as `ft_itoa`, for the identical reason: negating `INT_MIN` as a plain `int` is undefined behavior, and this function faces the exact same input. No heap allocation occurs anywhere in this family — every function writes directly to the file descriptor, so there is nothing to `free()` and no leak surface at all.

3.7 — The Linked List Engine

Nine functions built entirely on the two-field `t_list` struct from Part 1. This category is where pointer-to-pointer parameters (`t_list **lst`) first appear in the library — the mechanism that

lets a function reach back and modify the caller's own head pointer, not just a local copy of it.

ft_lstnew

CONCEPT: NODE CONSTRUCTION

```
t_list  *ft_lstnew(void *content)
{
    t_list  *new_node;

    new_node = (t_list *)malloc(sizeof(t_list));
    if (new_node == NULL)
        return (NULL);
    new_node->content = content;
    new_node->next = NULL;
    return (new_node);
}
```

TECHNICAL LOGIC

The atomic building block every other list function depends on. Note precisely what this function does *not* do: it does not copy `content` — it stores the pointer itself. The node and whatever `content` points to are two entirely separate allocations with two separate lifetimes, a distinction that becomes critical the moment `ft_lstdelete` enters the picture.

NORMINETTE & C STANDARDS

`new_node->next` is explicitly set to `NULL` rather than left uninitialized — every list-traversal function in this library trusts that a freshly constructed node's `next` is a well-defined sentinel, not garbage stack/heap memory.

MEMORY SECURITY

`malloc` failure returns `NULL` immediately, without touching `content` — correctly matching the subject's contract and leaving the caller's `content` pointer untouched (and therefore still safely freeable by the caller) on failure.

ft_lstadd_front

CONCEPT: POINTER-TO-POINTER MUTATION

```
void ft_lstadd_front(t_list **lst, t_list *new)
{
    if (lst == NULL || new == NULL)
        return ;
    new->next = *lst;
    *lst = new;
}
```

Before: `lst` → [A] → [B] → NULL
`new` → [X] → ???

Step 1: `new->next = *lst` (X now points at the OLD head, A)
`new` → [X] → [A] → [B] → NULL

Step 2: `*lst = new` (caller's head pointer itself is rewritten)
`lst` → [X] → [A] → [B] → NULL

TWO POINTER WRITES, ZERO TRAVERSAL — O(1) REGARDLESS OF LIST LENGTH.

TECHNICAL LOGIC

This function is the reason the parameter is `t_list **lst` (a pointer to the caller's pointer) rather than `t_list *lst`. If the parameter were a plain `t_list *`, reassigning it inside the function would only change a local copy — the caller's actual head-of-list variable would be completely untouched once the function returns. By passing the *address* of that variable, the line `*lst = new` reaches through the pointer and overwrites the caller's own variable directly. This two-step sequence — first link the new node forward to the old head, *then* repoint the head — must happen in exactly this order, or the old head's address would be lost after the first line overwrote it.

NORMINETTE & C STANDARDS

Both `lst` and `new` are checked for `NULL` in one compound condition before either is dereferenced — the mandatory guard-first shape, here protecting two independent pointers in a single line rather than two separate checks, since both failure conditions produce the identical no-op response.

MEMORY SECURITY

No allocation occurs in this function — it only rewires existing pointers, so there is no leak surface. The operation is O(1): the entire rest of the list is untouched and un-traversed, which matters for larger lists compared to `ft_lstadd_back`'s necessary O(n) traversal.

ft_lstsize / ft_lstlast

CONCEPT: LINEAR TRAVERSAL

```
int ft_lstsize(t_list *lst)
{
    int count;

    count = 0;
    while (lst != NULL)
    {
        count++;
        lst = lst->next;
    }
    return (count);
}
```

TECHNICAL LOGIC

Both functions share the same walking idiom: `lst = lst->next` inside a `while (lst != NULL)` loop. `ft_lstsize` counts steps; `ft_lstlast` instead checks `lst->next != NULL` and stops one node early, so it returns the last *node itself* rather than walking off the end into `NULL`. This distinction — testing the current node's `next` versus testing the current node itself — is precisely why `ft_lstlast`'s loop condition looks at `lst->next` instead of `lst`.

NORMINETTE & C STANDARDS

Notice the parameter `lst` is reassigned directly inside the loop rather than introducing a second "current" variable — this is safe and Norm-legal specifically because `lst` here is passed **by value** (a `t_list *`, not a `t_list **`): the function only ever holds a local copy of the caller's pointer, so walking it forward has zero effect on the caller's own list variable.

MEMORY SECURITY

Both correctly handle an empty list: `ft_lstsize(NULL)` returns `0` without entering the loop, and `ft_lstlast(NULL)` has an explicit early `if (lst == NULL) return (NULL)` guard before the loop even starts — without it, the first loop iteration's `lst->next` would dereference a `NULL` pointer.

ft_lstadd_back

CONCEPT: DELEGATED TRAVERSAL

```
void ft_lstadd_back(t_list **lst, t_list *new)
{
    t_list *last_node;

    if (lst == NULL || new == NULL)
        return ;
    if (*lst == NULL)
    {
        *lst = new;
        return ;
    }
    last_node = ft_lstlast(*lst);
    last_node->next = new;
}
```

TECHNICAL LOGIC

Two distinct cases, handled explicitly rather than hoping one code path covers both. If the list is currently empty (`*lst == NULL`), the new node *becomes* the list — assigned directly to `*lst`. Otherwise, the function delegates the traversal-to-the-end work entirely to `ft_lstlast` — a second demonstration of composition, this time reusing a same-category function rather than reimplementing the walk.

NORMINETTE & C STANDARDS

The empty-list case is written as its own guarded early return rather than folded into a single unified expression — this keeps the "list currently has zero nodes" case unambiguous and separately readable from the "append after the last node" case, even though both ultimately just assign a `next` or a head pointer.

MEMORY SECURITY

Reusing `ft_lstlast` here means the empty-list check must happen *first* and separately — calling `ft_lstlast(NULL)` would correctly return `NULL` per its own guard, but then `last_node->next = new` would dereference that `NULL`. The explicit early-return branch is exactly what prevents this.

ft_lstdeleteone / ft_lstclear

CONCEPT: OWNERSHIP-AWARE DEALLOCATION

```
void ft_lstdeleteone(t_list *lst, void (*del)(void *))
{
    if (lst == NULL || del == NULL)
        return ;
    del(lst->content);
    free(lst);
}

void ft_lstclear(t_list **lst, void (*del)(void *))
{
    t_list *current;
    t_list *next_node;

    if (lst == NULL || del == NULL)
        return ;
    current = *lst;
    while (current != NULL)
    {
        next_node = current->next;
        ft_lstdeleteone(current, del);
        current = next_node;
    }
    *lst = NULL;
}
```

```
*lst → [A] → [B] → [C] → NULL
      |
current —
```

```
iteration 1: next_node = B (SAVE the link before A is freed!)
             ft_lstdeleteone(A, del) → del(A.content); free(A)
             current = next_node → current now points at B
```

(repeat for B, then C)

```
after loop: *lst = NULL ← caller's pointer explicitly nulled,
                    preventing any future use-after-free
```

NEXT_NODE MUST BE SAVED BEFORE FREE(CURRENT) — FREEING FIRST WOULD

TECHNICAL LOGIC

`ft_lstdeleteone` frees exactly *one* node's content (via the caller-supplied `del` function, since only the caller knows what type `content` actually points to) and then the node struct itself — explicitly *not* touching `next`, per the subject's own contract.

`ft_lstclear` builds the full-list version on top of it, but must solve one specific ordering hazard: `current→next` cannot be read *after* `current` has been freed — that would be a use-after-free read. The fix is to capture `next_node = current→next` *before* calling `ft_lstdelone(current, del)`, so the chain's continuation is safely preserved in a local variable that survives the free.

NORMINETTE & C STANDARDS

`ft_lstclear` reuses `ft_lstdelone` rather than inlining `del(current→content); free(current)` directly — the same composition instinct applied consistently across every category in this library.

MEMORY SECURITY

The final line, `*lst = NULL`, is not cosmetic — it's what prevents a classic **dangling pointer** bug. After this function returns, every node the caller's variable used to point to has been freed; if the caller's own head pointer weren't explicitly reset, calling any list function on it again would operate on freed memory. Setting it to `NULL` converts a silent use-after-free hazard into a harmless, well-defined empty list.

ft_lstiter

CONCEPT: SIDE-EFFECTING TRAVERSAL

```
void ft_lstiter(t_list *lst, void (*f)(void *))
{
    if (lst == NULL || f == NULL)
        return ;
    while (lst != NULL)
    {
        f(lst->content);
        lst = lst->next;
    }
}
```

TECHNICAL LOGIC

The simplest of the callback-based list functions: walk every node, call `f` on its `content`, move on. `f` is expected to mutate `content` in place if it needs to (since `content` is itself a `void *`, whatever it points to can be modified through that pointer by a caller who knows the real type) — this function makes no copies and returns nothing.

NORMINETTE & C STANDARDS

Structurally near-identical to `ft_lstsize`'s traversal loop, with one added function call per iteration — a clean illustration of how a small, consistent traversal idiom scales across very different use cases just by varying what happens inside the loop body.

MEMORY SECURITY

No allocation, no freeing — purely a controlled way to apply an external effect across every node. The `NULL` check on `f` is what prevents a crash if a caller passes an empty callback by mistake.

ft_lstmap

CONCEPT: TRANSFORM-AND-ROLLBACK

```

t_list  *ft_lstmap(t_list *lst, void *(*f)(void *), void (*del)(void *))
{
    t_list  *new_list;
    t_list  *new_node;
    void     *new_content;

    if (lst == NULL || f == NULL || del == NULL)
        return (NULL);
    new_list = NULL;
    while (lst != NULL)
    {
        new_content = f(lst->content);
        new_node = ft_lstnew(new_content);
        if (new_node == NULL)
        {
            del(new_content);
            ft_lstclear(&new_list, del);
            return (NULL);
        }
        ft_lstadd_back(&new_list, new_node);
        lst = lst->next;
    }
    return (new_list);
}

```

```

old: [1]→[2]→[3]→NULL

step i=0: new_content = f(1)      → e.g. 1
          new_node = ft_lstnew(new_content)
          IF new_node alloc FAILS:
              del(new_content)      ← free the orphaned content
              ft_lstclear(&new_list, del) ← free every node built SO FAR
              return NULL           ← leave nothing half-built

          ELSE: ft_lstadd_back(&new_list, new_node)

new: [1]→[4]→[9]→NULL      (after all 3 iterations succeed)

EVERY ALLOCATION IS CHECKED MID-LOOP — A FAILURE ANYWHERE UNWINDS

```

TECHNICAL LOGIC

The most sophisticated function in the library, and the one that ties every other list function together: it applies `f` to each node's content to produce `new_content`, wraps that in a fresh node via `ft_lstnew`, and appends it to a brand-new list via `ft_lstadd_back` — leaving the original `lst` completely untouched. This is a genuine

map operation in the functional-programming sense: input list unmodified, new list returned.

NORMINETTE & C STANDARDS

Reuses three other library functions in nine lines of actual logic (`ft_lstnew` , `ft_lstadd_back` , `ft_lstclear`) — the single clearest example in the entire codebase of why the composition discipline established back in `ft_isalnum` matters at scale: without those three building blocks already existing and already correct, this function would need to reimplement node allocation, list appending, *and* cleanup-on-failure all inline, which would blow well past the 25-line limit.

MEMORY SECURITY

This is the only function in the library with a genuine **rollback path**, and it's worth tracing exactly why each line of the failure branch exists. If `ft_lstnew` fails partway through the loop, `new_content` was already successfully produced by `f` but has no node to live in — so `del(new_content)` frees it directly, by hand, before it becomes unreachable. Then `ft_lstclear(&new_list, del)` frees every node *successfully* built in earlier iterations — without this line, a failure on, say, the fourth node out of ten would silently leak the three nodes already built. Only after both cleanup steps does the function return `NULL` , guaranteeing that on failure, the function's net heap footprint is exactly zero — no partial, half-built list is ever handed back to the caller.